

Online Review Sentiment Analysis: Flipkart and Twitter

Deliverable 4: Result Interpretation and Report

Name	Tamim Ahmed
Student ID	905253015
Group	Individual submission
Date	3 October 2026
GitHub repository	[add the repository link after pushing]

1. Objective and data

The aim of this project is to build a pipeline that reads a raw product review or tweet, labels it **positive**, **neutral** or **negative**, and then uses those labels to describe how customers feel about an online store's products and services.

Rather than use the class sample file, I wrote my own generator (`src/generate_dataset.py`) to produce a new synthetic dataset. The sample file is a useful demo but too easy to learn from: its 1,000 rows contain only 497 distinct sentences, and star ratings are spread across 1 to 5 for both positive and negative labels. My generator combines product features, service aspects (delivery, packaging, price, customer care, returns), openers, closers and platform styles, and deliberately includes harder cases: **negation** ("not worth the money"), **mixed opinions**, **sarcasm**, **wait-and-see** posts with no opinion yet, and about **2% flipped labels** to mimic disagreement between human annotators. To test whether the trend analysis can recover real events, it also builds in three: a festive-sale delivery crunch (October–November 2025), a smartwatch launched in February 2026 with a battery and app-sync bug that a firmware update fixed in May, and monsoon delivery delays (July–August 2026).

The final dataset has **2,000 unique texts** about 12 products, dated October 2025 to September 2026: 1,207 Flipkart reviews and 793 tweets. 46% are positive, 30% negative and 24% neutral. Star ratings follow sentiment (average 4.32 for positive, 2.88 for neutral and 1.68 for negative reviews) but are never used as a model input, because the rating would give the answer away.

2. The pipeline in brief

Preprocessing. Each text is lowercased; URLs and @mentions are removed; hashtags keep their word (#fail becomes "fail"); emojis are turned into the tokens `emo_pos`, `emo_neg` or `emo_neu` rather than deleted; contractions are expanded ("doesn't" becomes "does not"); elongations are shortened ("sooo"); and the text is tokenised with NLTK before stopwords removal. The important choice is that I **kept negations and contrast words**. NLTK's English stopwords list contains "not", "no", "never" and "but", so the class-demo cleaner turns "not good at all" into "good". In five-fold cross-validation on the training data, my cleaner raised macro-F1 from 0.933 to 0.947.

Features and split. The data are split 80/20 with stratification (1,600 training and 400 test reviews), so each class keeps its share in both parts. I chose the TF-IDF settings by cross-validation on the training set only. Unigrams alone scored 0.946 at every size because there are only 431 distinct words. Adding **bigrams** lifted the score to 0.947 once at least 2,000 features were allowed, because phrases such as "not worth", "very disappointed" and "three stars" carry meaning that single words do not. Trigrams lowered the score (0.904–0.932) by adding about 2,000 mostly one-off phrases. The bigram vocabulary is 2,075 terms, so **max_features = 2,000** keeps nearly all of it at no cost while capping the size if more data are added.

Models. I trained four classifiers, each inside a scikit-learn Pipeline with its own vectoriser so that cross-validation never sees test vocabulary. **Logistic Regression** is a fast linear model with interpretable weights and probability outputs. **Multinomial Naive Bayes** is the classic text baseline. **Linear SVM** is a max-margin linear model that usually performs best on sparse, high-dimensional TF-IDF data. **Random Forest** is a non-linear ensemble, included to test whether word interactions help. The **VADER** sentiment lexicon, which needs no training, serves as a baseline.

3. Model comparison

Table 1. Results on the same held-out test set (n = 400). Precision, recall and F1 are weighted averages; macro-F1 weights the three classes equally. The last column is five-fold cross-validation on the training set.

Model	Accuracy	Precision	Recall	F1 (w)	F1 (macro)	CV macro-F1
Linear SVM	0.960	0.960	0.960	0.960	0.954	0.961 ± 0.019
Random Forest	0.945	0.945	0.945	0.945	0.939	0.933 ± 0.020
Logistic Regression	0.935	0.935	0.935	0.934	0.928	0.947 ± 0.019
Multinomial NB	0.903	0.902	0.903	0.902	0.890	0.899 ± 0.026
VADER lexicon (no training)	0.658	0.649	0.658	0.644	0.610	n/a

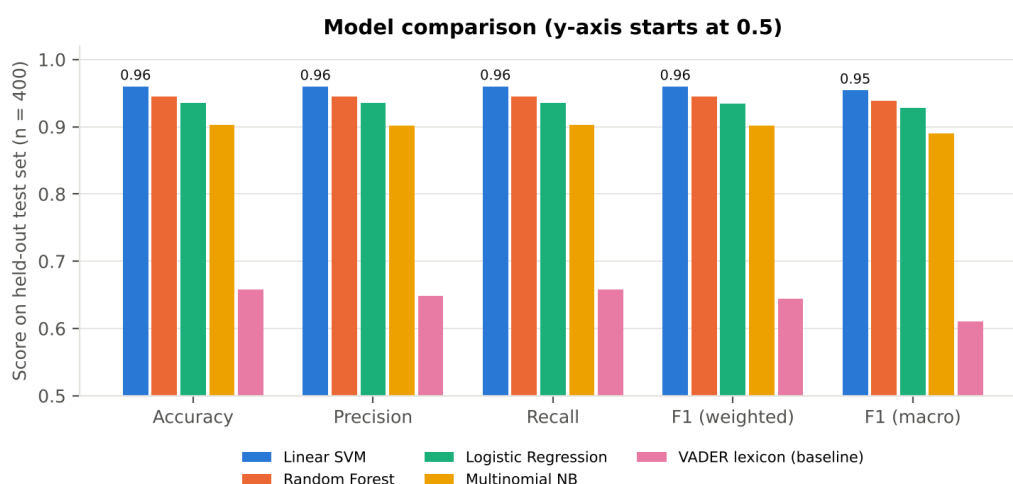


Figure 1. All five approaches on five metrics (y-axis starts at 0.5).

All four trained models perform far better than VADER (accuracy 0.658), which shows the value of learning this domain's vocabulary, such as "returning", "three stars" or "#refundplease". **Linear SVM is best on every metric:** 0.960 accuracy, 0.954 macro-F1, and the highest cross-validation score (0.961 ± 0.019), so its first place does not depend on one lucky split. The gaps between SVM, Random Forest and Logistic Regression are small (1.5–2.5 points, or 6–10 test reviews), and Logistic Regression beats Random Forest in cross-validation, so the order of those two is within noise. Naive Bayes trails because its word-independence assumption copes poorly when positive and negative words appear in the same review.

I recommend the Linear SVM. It has the highest macro-F1, which matters because the smallest class (neutral) is also the hardest; its cross-validation is the most consistent; and it trains in under a second. If a business needs a confidence score for each review, for example to send uncertain cases to a person, Logistic Regression is a close second and outputs probabilities directly.

4. What the confusion matrix shows

Confusion matrix: Linear SVM

Actual	negative	117	3	0
	neutral	3	86	6
	positive	0	4	181
		negative	neutral	positive
		Predicted		

Figure 2. Confusion matrix, Linear SVM (test set, n = 400).

Table 2. Test-set accuracy by type of text: Linear SVM vs. VADER.

Type of review	n	SVM	VADER
Plain opinion	243	98%	75%
Mixed (pros and cons)	79	100%	53%
Negation	27	96%	89%
Wait-and-see (neutral)	28	100%	46%
Sarcasm	12	100%	0%
Flipped label (noise)	11	0%	27%

The model almost never confuses opposite polarities: **no positive review was predicted negative, and no negative review was predicted positive.** Its mistakes are about whether an opinion is present at all. **Neutral is the hardest**

class (recall 0.905, F1 0.915, against about 0.97 for the other two). The single most common error is **neutral predicted as positive** (6 of 95). There are three plausible reasons. Neutral has no vocabulary of its own: it is defined by the absence of a strong opinion, or by a balance of both ("Pros: ... Cons: ..."), so it borrows words from both sides. It is also the smallest class (24%). And it is the most subjective label, where human annotators disagree most.

The error analysis supports this. **Eleven of the SVM's 16 errors are the deliberately flipped labels**. For example, "Build quality is solid and reliable." is labelled neutral and predicted positive, so the model is right and the label is wrong. The five genuine errors are mostly short reviews in which a procedural, neutral-sounding sentence ("Requested an exchange for a different size", "Support said a technician will visit") outweighs a brief opinion such as "Total waste of money". The 100% scores for sarcasm and mixed reviews in Table 2 should not be over-read: the generator's sarcastic sentences reuse a small set of patterns that the model memorised. VADER got none of the 12 sarcastic tweets right, which is closer to how hard sarcasm really is.

5. Sentiment trends

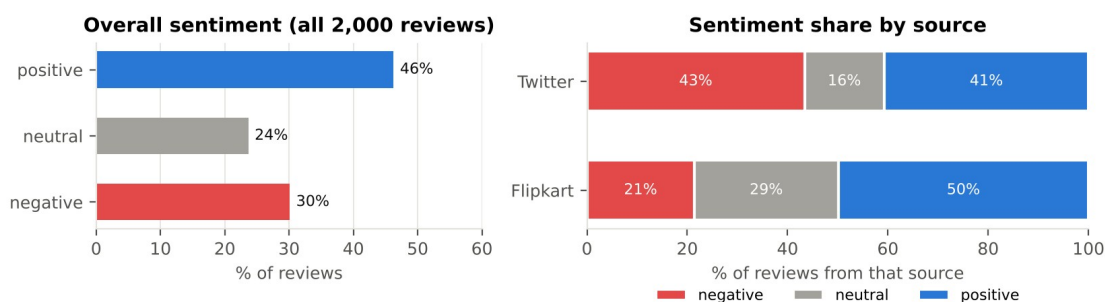


Figure 3. Overall sentiment and sentiment by source.

Overall and by source. Customers are broadly satisfied (46% positive) but a sizeable minority (30%) is unhappy. The two platforms behave very differently. **Twitter is twice as negative as Flipkart** (43% against 21%) and far less neutral (16% against 29%). People tend to tweet when something goes very well or very badly, and they tag the brand to get a reply, while marketplace reviews are often prompted after delivery and include measured "it's okay" verdicts. Customer service accounts for **28% of Twitter complaints** but only 5% on Flipkart, which suggests Twitter is where customers go after support has failed them.

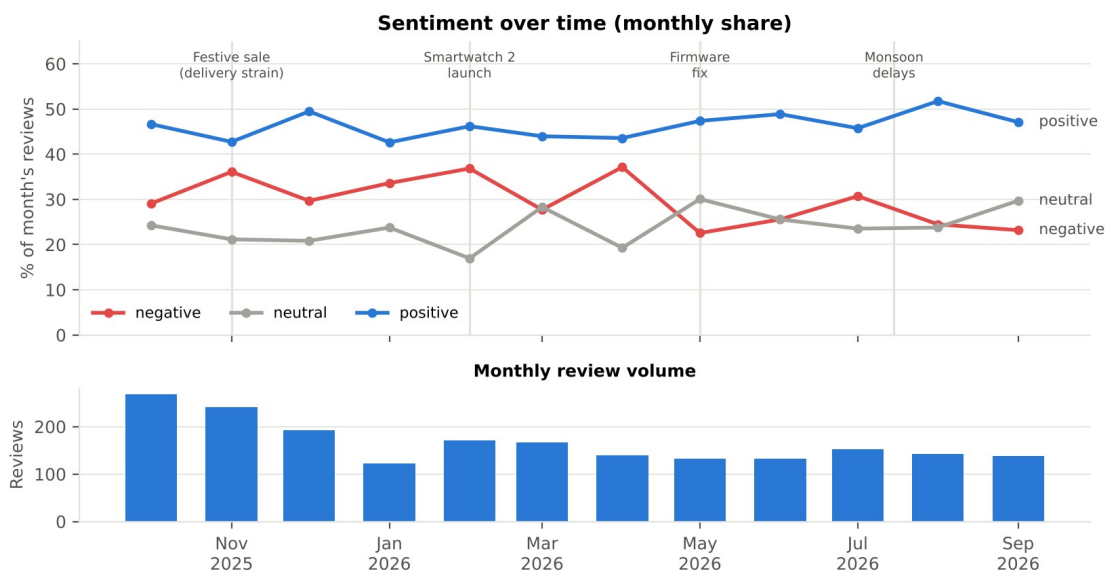


Figure 4. Monthly sentiment share (top) and review volume (bottom).

Over time. Review volume peaks during the festive sale (268 reviews in October and 241 in November), and so does negativity (36.1% negative in November). **Delivery causes 44% of negative reviews in the sale and monsoon months, compared with 8% in the other months**, which explains the second rise in July (30.7%). The February–April rise is almost entirely the smartwatch: without it, March would have been the calmest month of the year (19% negative). Apart from the July monsoon rise, sentiment is at its best from May onwards. Each month has only 120–270 reviews, so a single month's share can move by 3–4 points by chance; patterns that last several months are more reliable than any single point.

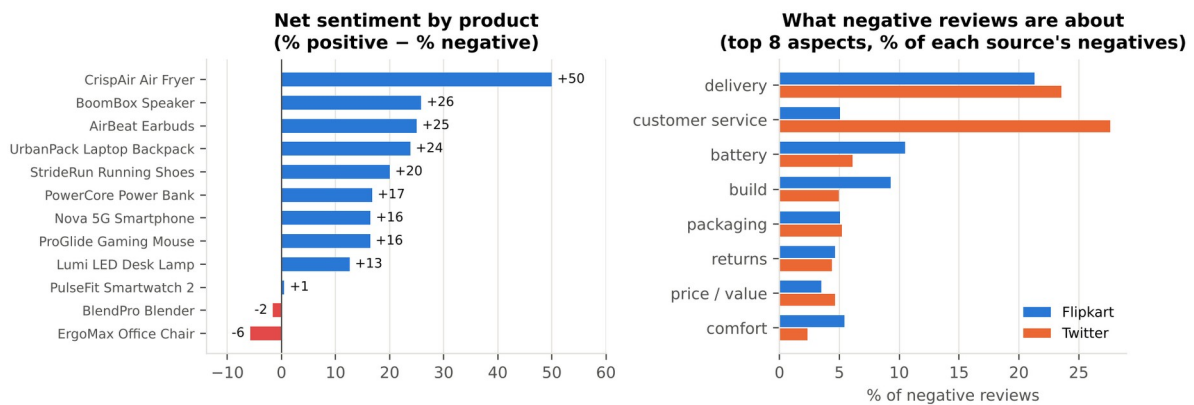


Figure 5. Net sentiment (% positive – % negative) by product, and the topics of negative reviews.

By product and topic. Net sentiment ranges from +50 for the CrispAir Air Fryer to -6 for the ErgoMax Office Chair; the BlendPro Blender (-2) is the only other product below zero. Delivery is the leading complaint overall (first on Flipkart and second on Twitter, behind customer service); battery life and build quality are the main product complaints on Flipkart. The smartwatch's overall score (+1) hides two very different products (Figure 6): before the firmware fix, 76% of its reviews were negative (net -64); afterwards, only 14% were (net +43).

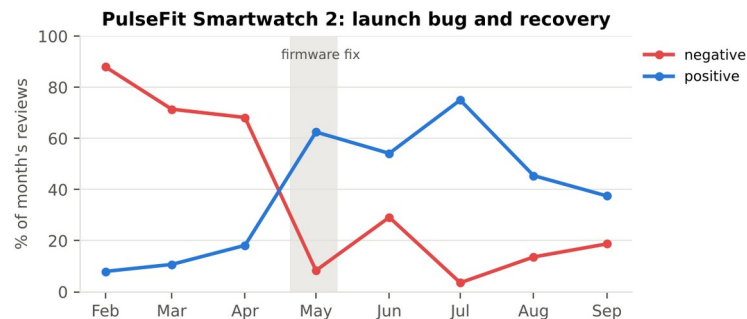


Figure 6. PulseFit Smartwatch 2: monthly share of positive and negative reviews (20–30 reviews per month).

6. Limitations and how to improve

- **Synthetic, template-based data inflate the scores.** This is the most important limitation. I tested the trained models on **36 reviews I wrote by hand** in phrasing the generator never uses. Accuracy fell from 96% to **47% for the SVM** (42–50% across the four models), below the untrained VADER lexicon (61%). The SVM recognised only 3 of 12 positive reviews, because phrases such as "saved my back" or "bone dry" never appear in training. Some of the most heavily weighted terms, such as "honestly" and "go" (from "Go for it"), are artefacts of the templates. The class sample shows the same problem in a more extreme form: the pipeline scores 100% there because half its rows are duplicates.
- **Sarcasm and context.** A bag of words and bigrams ignores longer word order, so real sarcasm ("Oh lovely, a 'brand new' speaker with someone else's fingerprints") cannot be memorised the way the templated version was.
- **Subjective neutral class and label noise.** Only 2% of labels were flipped, yet they caused 11 of 16 errors. In real data, people often disagree on whether a review is neutral or mildly positive, so the measured accuracy is limited by label quality as much as by the model.
- **Class imbalance and language.** Neutral makes up only 24% of the data; stratification and macro-F1 reduce this problem but do not remove it. The model also handles English only, while many customers in South Asia write in Bangla, Hindi or romanised mixes ("product ta valo na").

Improvement. The most useful next step is to **train on a few thousand real, human-labelled reviews** and replace TF-IDF with a **fine-tuned pre-trained transformer** (DistilBERT, or multilingual XLM-RoBERTa for code-mixed text), keeping a held-out set of real reviews as the test. A transformer recognises paraphrases such as "saved my back" and takes word order into account. A cheaper interim step is to combine the SVM with VADER and use the lexicon's verdict when the SVM's decision margin is small.

7. Business implications

If I were advising this retailer, the analysis would point to six actions:

- **Fix delivery first.** It is the most common complaint overall and accounts for almost half of all complaints in peak months. Add courier capacity and set realistic delivery dates before festive sales and the monsoon, and send delay notices before customers have to ask.
- **Treat Twitter as a customer-service channel.** Twitter is 43% negative and these complaints are public. Monitor brand mentions in real time and set a response target (for example, under one hour), since more than a quarter of Twitter complaints are about support itself.
- **Investigate the two products with negative net sentiment.** For the ErgoMax chair (assembly, missing screws, build) and the BlendPro blender (jar quality), raise the issues with suppliers, improve assembly instructions and check incoming stock.
- **Watch new launches closely.** The smartwatch went from net -64 to +43 once the bug was fixed. A weekly dashboard that flags any product whose negative share jumps would have caught the problem in its first week rather than after three months of damage.
- **Build on strengths.** The air fryer (+50), speaker and earbuds are clear favourites. Feature them in campaigns and ask satisfied buyers to leave reviews.
- **Use the model for triage, not as the final word.** Given the generalisation gap, retrain it on the company's real reviews and have a person check low-confidence cases before acting on its numbers.

8. Conclusion

A TF-IDF and Linear SVM pipeline, with negation-aware preprocessing, classifies this synthetic review data with 96% accuracy and recovers all three business events built into it: the sale-season and monsoon delivery problems, and the smartwatch launch and recovery. Neutral reviews are the hardest to classify, and most remaining errors come from noisy labels. The hand-written challenge set is a reminder that strong scores on synthetic data do not guarantee strong scores on real customers' language. The next step is real, labelled data and a context-aware model. The code, data, notebook and all figures are in the GitHub repository, which can be run from start to finish with one command.